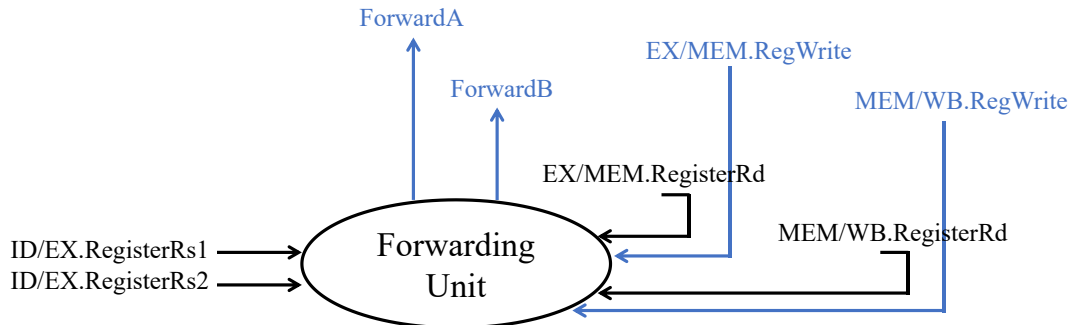


典型冒险整理（教材）

一、 数据冒险

① Forwarding Unit:



EX/MEM 段的冒险:

```
if(EX/MEM.RegWrite
  and (EX/MEM.RegisterRd  $\neq$  0)
  and (EX/MEM.RegisterRd = ID/EX.RegisterRs1)) ForwardA=10
if(EX/MEM.RegWrite
  and (EX/MEM.RegisterRd  $\neq$  0)
  and (EX/MEM.RegisterRd = ID/EX.RegisterRs2)) ForwardB=10
```

MEM/WB 段的冒险:

```
if(MEM/WB.RegWrite
  and (MEM/WB.RegisterRd  $\neq$  0)
  and not (EX/MEM.RegWrite and (EX/MEM.RegisterRd  $\neq$  0)
    and (EX/MEM.RegisterRd = ID/EX.RegisterRs1))
  and (MEM/WB.RegisterRd = ID/EX.RegisterRs1)) ForwardA=01
if(MEM/WB.RegWrite
  and (MEM/WB.RegisterRd  $\neq$  0)
  and not (EX/MEM.RegWrite and (EX/MEM.RegisterRd  $\neq$  0)
    and (EX/MEM.RegisterRd = ID/EX.RegisterRs2))
  and (MEM/WB.RegisterRd = ID/EX.RegisterRs2)) ForwardB=01
```

注: i. RegisterRd $\neq$ 0 的判定, 是因为 x0 寄存器恒为零, 不应该实现前递;

ii. not 语句是为了避免如下情况, 需要前递的是第二句的 rd (即 x5)

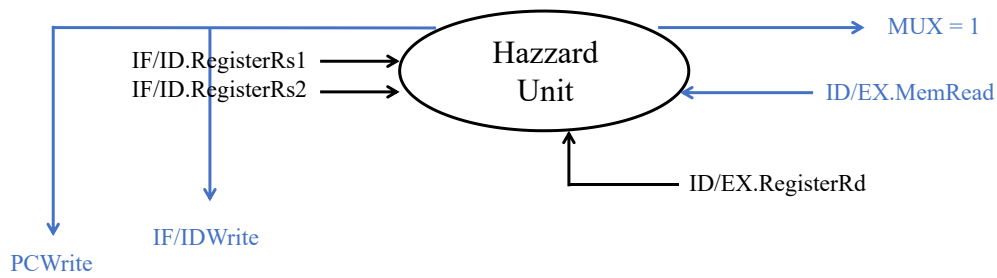
add x5, x6, x7

add x5, x6, x8

add x10, x5, x9

iii. ForwardB 信号需要两级 MUX 才能维持结构不变。

② Hazzard Detecting Unit:



```

if(ID/EX.MemRead
  and (ID/EX.RegisterRd  $\neq$  0)
  and ((IF/ID.RegisterRs1 = ID/EX.RegisterRd)
    or (IF/ID.RegisterRs2 = ID/EX.RegisterRd)))
{PCWrite = 0; IF/IDWrite = 0; MUX = 1;}

```

注：i. PCWrite = 0 是希望 other 之后的指令不进入队列；
 ii. IF/IDWrite = 0 是希望 other 指令不要在 IF/ID 寄存器中代替 use 指令；
 iii. MUX = 1 是希望清空已经保存在 ID/EX 寄存器中 use 的信息，这样可以实现停顿一个周期；

刷新
判定load-use

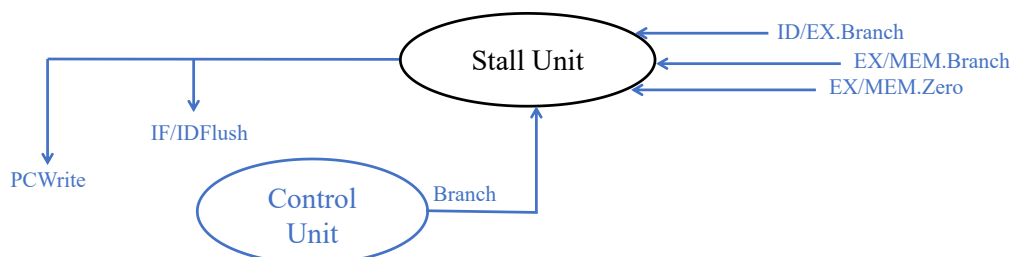
	Clk1	Clk2	Clk3	Clk4	Clk5	Clk6	Clk7	Clk8
lw	IF	ID	EX	MEM	WB			
use		IF	ID	○	○	○		
other			IF	ID	EX	MEM	WB	
				IF	ID	EX	MEM	WB

注：表示上，可以选择把 use 和 other 指令进行向上压缩！

二、 控制冒险 —— 讨论默认 PCSrc 指令在 MEM 阶段生成

控制冒险 { 软件处理方法 { 时间槽插入NOP语句（编译器或者汇编语言完成）
 时间槽插入时间槽指令 注：把分支语句后侧的三条指令称为时间槽
 硬件处理方法 { 不预测方式
 预测 + 错误处理
 在ID阶段补充相等比较

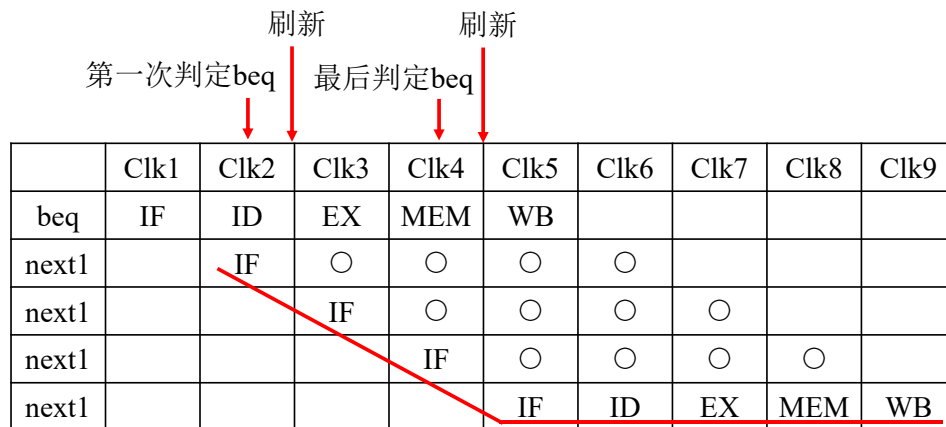
① Stall Unit:



if(Branch
or ID/EX.Branch
or (EX.MEM.Branch and not EX/MEM.Zero)) PCWrite = 0;

if(Branch
or ID/EX.Branch
or (EX.MEM.Branch)) IF/IDFlush = 0;

注: i. 考虑不跳转的情况, 在 Clk4 中, beq 指令已经解读出 PCSrc = 1(即跳转), 所以在 Clk4 和 Clk5 之间的上升沿, 需要不让 PC 刷新!

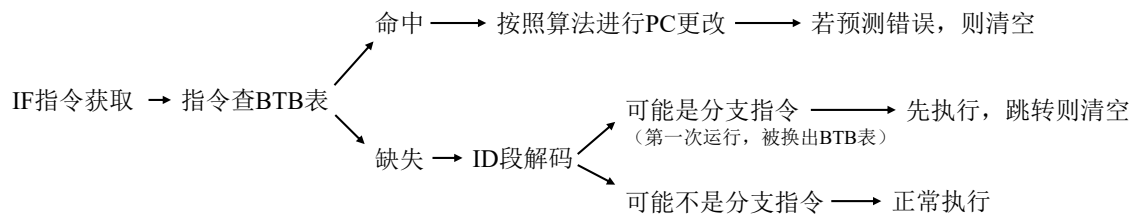


ii. 考虑跳转的情况, 在 Clk4 和 Clk5 之间的上升沿, 需要让 PC 刷新!



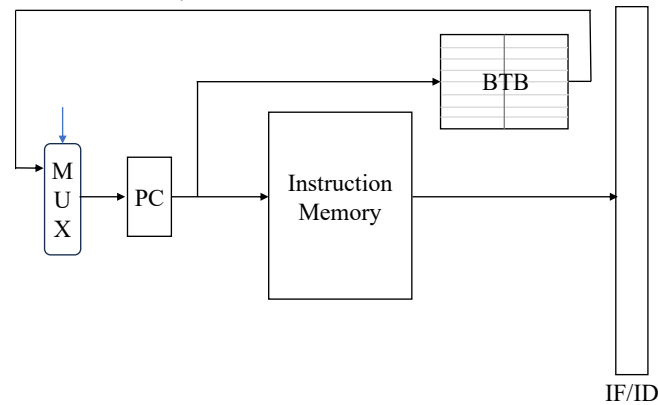
iii. 明显地, 停顿处理固定浪费 3 个时钟周期;

② Prediction:

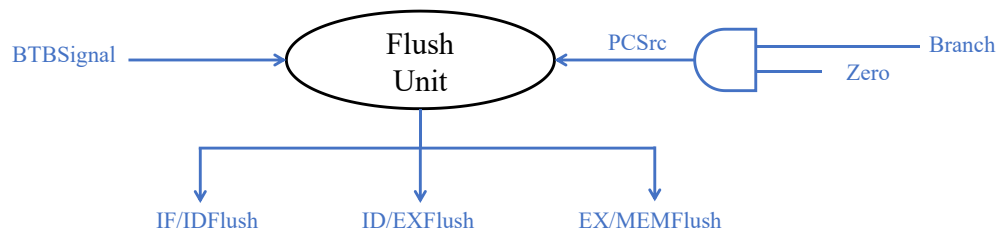


注：i. 除了静态预测不跳转可以不用 BTB 表之外，其余都需要查表！

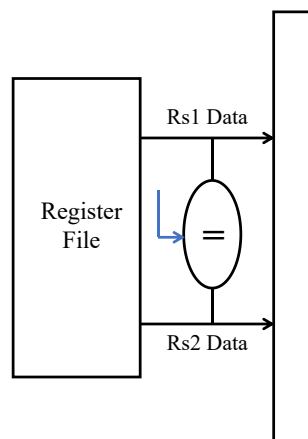
ii. 以下步骤可以在 IF 阶段完成。



iii. 本图展示了 Flush 信号的逻辑，他和 BTBSignal 有关，即是否预测跳转有关，也和 PCSrc 有关，即与是否满足跳转有关，如果需要清空信号，则将三个流水线寄存器清空，特别注意 PCWrite 信号为 1。



③ 提前分支判定到 ID 阶段:



```
if(Branch and (Rs1 Data = Rs2 Data))
{PCWrite =1; IF/IDFlush=1;}
```

注：这是停顿的处理方式，但是真实情况是如果把 beq 指令调整到 ID 阶段，还会产生数据冒险：

如果 pre1 是 load 指令，需要在 beq 前停顿两次；

如果 pre2 是 load 指令，需要在 beq 前停顿一次；

如果 pre1 是 add 指令，需要在 beq 前停顿一次；

可以通过流水线进行分析。

	Clk1	Clk2	Clk3	Clk4	Clk5	Clk6	Clk7	Clk8	Clk9
pre4	IF	ID	EX	MEM	WB				
pre3		IF	ID	EX	MEM	WB			
pre2			IF	ID	EX	MEM	WB		
pre1				IF	ID	EX	MEM	WB	
beq					IF	ID	EX	MEM	WB

易错点：

① sw rs2, offset(rs1);

② 考虑误解读的可能性，特别是 SBtype 的 rd 字段；

Name (Field Size)	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	Comments
R-type	funct7	rs2	rs1	funct3	rd	opcode	Arithmetic instruction format
I-type	immediate[11:0]		rs1	funct3	rd	opcode	Loads & immediate arithmetic
S-type	immed[11:5]	rs2	rs1	funct3	immed[4:0]	opcode	Stores
SB-type	immed[12,10:5]	rs2	rs1	funct3	immed[4:1,11]	opcode	Conditional branch format
UJ-type	immediate[20,10:1,11,19:12]				rd	opcode	Unconditional jump format
U-type	immediate[31:12]				rd	opcode	Upper immediate format